SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

*I **KANMANI K B (192524306)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Problem

We have:

- One **4-gallon jug**
- One **3-gallon jug**
- No measuring marks
- Unlimited water from a pump

Goal: Get exactly **2 gallons in the 4-gallon jug**.

State Representation

Represent a state as:

(x, y)

where:

- x = amount of water in the 4-gallon jug
- y = amount of water in the 3-gallon jug

Initial state:

(0, 0)

Goal state:

(2, y), where the 4-gallon jug contains exactly 2 gallons.

Steps

Step	Action	4-Gallon	3-Gallon
1	Start	0	0

2	Fill 3-gallon jug	0	3
3	Pour 3-gallon into 4-gallon	3	0
4	Fill 3-gallon jug again	3	3
5	Pour 3-gallon into 4-gallon until 4-gallon is full	4	2
6	Empty the 4-gallon jug	0	2
7	Pour the remaining 2 gallons into 4-gallon jug	**2**	0

Final Answer

The 4-gallon jug contains exactly 2 gallons of water.

State-Space Solution

(0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0)

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

i. **What are the percept's for this agent?**
ii. **Characterize the operating environment.**
iii. **What are the actions the agent can take?**
iv. **How can one evaluate the performance of the agent?**
v. **What sort of agent architecture do you think is most suitable for this agent? Why?**

A Mars Rover is an intelligent agent that explores the surface of Mars, collects information and analyzes samples.

i. What are the percepts for this agent?

Percepts are the information received by the rover through its sensors.

The Mars Rover can perceive:

- Images of the Martian surface
- Temperature
- Pressure
- Atmospheric conditions
- Rocks and soil

- Chemical composition of samples
- Distance from obstacles
- Rover's location
- Battery level
- Terrain conditions
- Possible hazards

Example: If the rover's camera detects a large rock in front of it, that becomes a percept.

ii. Characterize the operating environment

The Mars Rover environment can be characterized as:

Property	Mars Rover Environment
Observable	**Partially observable**
Deterministic	**Partially deterministic / stochastic**
Episodic	**Sequential**
Static	**Dynamic**
Discrete/Continuous	**Continuous**
Number of agents	**Single-agent**
Known/Unknown	**Partially unknown**

Explanation

- **Partially observable:** The rover cannot see the entire Mars surface.
- **Stochastic:** Outcomes may be uncertain because of terrain and environmental conditions.
- **Sequential:** One action affects future actions.
- **Dynamic:** Environmental conditions can change.
- **Continuous:** Position, temperature, distance, etc. have continuous values.
- **Single-agent:** The rover generally performs its mission independently.
- **Unknown:** The rover may encounter previously unknown terrain or samples.

iii. What actions can the agent take?

The Mars Rover can:

1. Move forward.
2. Move backward.
3. Turn left.
4. Turn right.
5. Stop.

6. Take photographs.
7. Scan rocks and soil.
8. Collect samples.
9. Analyze samples.
10. Communicate with the control station.
11. Avoid obstacles.
12. Recharge/manage its battery.

iv. How can we evaluate the performance of the agent?

The rover's performance can be evaluated based on:

- Amount of area explored
- Number of useful samples collected
- Accuracy of sample analysis
- Scientific discoveries made
- Distance successfully travelled
- Number of obstacles avoided
- Energy/battery efficiency
- Communication success
- Mission completion
- Safety of the rover

Performance measure

High scientific discoveries + successful exploration + minimum energy consumption + safe operation = Good performance.

v. Most suitable agent architecture

Model-Based Utility-Based Agent

This is suitable because the rover:

- Maintains information about its environment.
- Cannot observe everything directly.
- Needs to make decisions under uncertainty.
- Has multiple goals such as exploration, sample collection and safety.
- Must select the best action among alternatives.

Why?

For example, suppose the rover finds two possible locations:

- Location A has interesting rocks but is dangerous.
- Location B has fewer interesting rocks but is safe and nearby.

A utility-based agent can compare the **scientific value, danger, distance and battery consumption** and choose the better action.

Conclusion: A **model-based utility-based intelligent agent** is most suitable for a Mars Rover.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Problem

The objective is to place **8 queens on an 8 × 8 chessboard** such that:

No two queens attack each other.

A queen can attack another queen if they are in:

- The same row
- The same column
- The same diagonal

Initial State

An empty chessboard.

Goal State

A configuration containing 8 queens where:

- No two queens share a row.
- No two queens share a column.
- No two queens share a diagonal.

State Representation

A state can be represented as:

[r1, r2, r3, r4, r5, r6, r7, r8]

where ri represents the row position of the queen in column i.

For example:

[1, 5, 8, 6, 3, 7, 2, 4]

represents one valid solution.

Search Space

Initially:

0 queens placed

Then:

1 queen → 2 queens → 3 queens → ... → 8 queens

If we place one queen in each column, there are initially:

$8^8 = 16{,}777{,}216$

possible arrangements before eliminating invalid states.

If we ensure one queen per row and column, the possibilities reduce to:

$8! = 40{,}320$

arrangements.

Operators

The operator is:

Place a queen in an empty square.

Constraints

For queens at (r1,c1) and (r2,c2):

They must satisfy:

$r1 \neq r2$

$c1 \neq c2$

$|r1-r2| \neq |c1-c2|$

The third condition prevents diagonal attacks.

Search Strategies

Possible search strategies include:

- Breadth First Search
- Depth First Search
- Backtracking
- Hill Climbing
- Genetic Algorithm

Suitable strategy

Backtracking / Depth First Search is commonly suitable.

The algorithm places one queen at a time. If a queen creates a conflict, it goes back and tries another position.

Example valid solution

One solution is:

[1, 5, 8, 6, 3, 7, 2, 4]

This means:

Column	Queen Row
1	1
2	5
3	8
4	6
5	3
6	7
7	2
8	4

No two queens attack each other.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Problem

A customer wants to travel from one location to another using the OLA application.

The customer can select different cab types such as:

- Mini
- Micro
- Sedan
- Shared
- Prime

The agent must help select a suitable cab.

Type of Problem-Solving Agent

A **Goal-Based Agent** can be used.

The main goal is:

Reach the destination successfully.

However, since the customer has choices based on comfort, cost and travel time, a **Utility-Based Agent** is even more suitable.

Suitable answer:

Utility-Based Problem-Solving Agent

Why?

The agent can compare:

- Cost
- Distance
- Travel time
- Cab availability
- Customer comfort
- Cab type
- Traffic conditions

and select the cab that gives the **highest utility** according to the customer's requirements.

Problem Formulation

Initial State

Customer is at the starting location.

Goal State

Customer reaches the destination.

Actions

- Enter source
- Enter destination
- Select cab type
- Check available cabs
- Compare fare
- Book cab
- Travel
- Reach destination

Cost

Cost can include:

Fare + Travel time + Distance

Pseudocode

START

Input source location

Input destination location

Find available cabs

For each available cab:

Get cab type

Get estimated fare

Get estimated travel time

Get distance

Get comfort rating

Calculate utility for each cab

Select the cab with highest utility

Display cab details

Display estimated fare

Display estimated arrival time

Ask customer for confirmation

IF customer confirms:

Book the selected cab

Track cab

Navigate to destination

Display "Destination Reached"

ELSE:

Allow customer to select another cab

END

Example

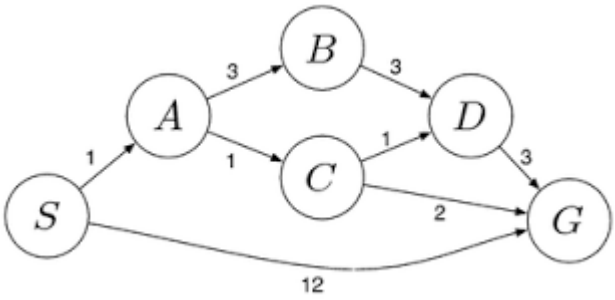
Suppose:

Cab	**Fare**	**Comfort**	**Time**
Micro	₹150	Medium	30 min
Sedan	₹250	High	25 min
Prime	₹350	Very High	22 min
Shared	₹100	Low	40 min

The utility-based agent evaluates these options according to the customer's preferences and recommends the most suitable cab.

5**. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.**

The delivery network is represented as a weighted graph:



Problem

A logistics company wants to transport a package from starting warehouse **S** to destination warehouse **G** at the **minimum total cost**.

Each connection between warehouses has a travel cost.

This is a **path-finding problem in a weighted graph**.

Problem Formulation

- **Initial state:** S
- **Goal state:** G
- **States:** Warehouses
- **Actions:** Travel from one warehouse to a connected warehouse
- **Path cost:** Sum of all route costs
- **Goal:** Find the path from S to G with minimum total cost.

Uniform Cost Search

UCS always expands the node with the **lowest cumulative path cost**.

It uses a **priority queue**.

Formula

If:

S → A → B → G

and the costs are:

- S → A = 2
- A → B = 3

- B → G = 4

Then:

Total cost = 2 + 3 + 4 = 9

UCS compares this cost with other possible paths and selects the path having the smallest total cost.

UCS Pseudocode

```
UCS(Graph, Start, Goal)
Create priority queue PQ
Insert Start into PQ with cost 0
Create visited set
WHILE PQ is not empty:
    Remove node with minimum path cost
    IF node is Goal:
        Return path and total cost
    IF node is already visited:
        Continue
    Mark node as visited
    FOR each neighbor of node:
        Calculate new cost
        Insert neighbor into PQ
       with the new cumulative cost
Return "No path found"
```

Possible paths include:

S → A → C → G

Cost:

2 + 3 + 4 = 9

Another path:

S → B → C → G

Cost:

5 + 2 + 4 = 11

Therefore, UCS selects:

S → A → C → G

with minimum cost:

9

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem